Code Formatting and Linter Suppressions

Independent research report | Public draft, November 2026

1. Introduction

Can formatting Python code change which lint warnings are suppressed, even when its executable syntax stays the same? I tested this question using Black and Ruff as formatters and Ruff as the linter.

A comment such as `# noqa: F821` tells Ruff to hide an undefined-name warning. Formatting can move that comment or put more code on the line it covers. A warning may then disappear from normal lint output even though the code that triggered it is still present. Formatting can also separate code from a comment that previously suppressed its warnings.

I compared warning visibility before and after formatting. Checking the number of warnings alone would not be enough: a warning could move, become hidden, or be replaced by another warning. The comparison therefore checks the underlying warnings and matches their source occurrences before attributing a change to suppression.

The experiment has two parts. A controlled suite tests specific source layouts and directive placements. A separate sample contains files from public Python repositories. The controlled cases show when changes can occur; their proportions do not estimate how common those changes are in ordinary projects.

2. Related Work

Hu et al. [1] studied warning suppressions and their evolution across project histories. Their treatment of line-splitting cases shows why a changed warning count does not necessarily mean a new warning. Their setting differs from isolating a formatter's effect on an otherwise fixed input.

Li and Yang [2] studied warning tracking across revisions. That work supports treating correspondence as a problem to check, rather than assuming that warnings on the same line are the same warning. McKeeman's account of differential testing [3] provides a related testing idea: differences between comparable systems identify cases worth investigating.

The general formatter/directive interaction is already documented. Black's FAQ discusses moved linter comments [4]. Ruff's formatter documentation gives an example where collapsing a list broadens the coverage of a `noqa` comment [5]. I do not claim to have discovered this possibility. This project supplies a paired experiment, saved diagnostic evidence and a separate natural-source check.

3. Methodology

3.1 Four diagnostic checks

The experiment used CPython 3.11.9, Black 25.1.0 and Ruff 0.11.13 on Windows. These are fixed historical versions. Formatting used line length 88, target Python 3.11 and no preview mode. The selected rules were F401 (unused import), F821 (undefined name) and F841 (unused local variable). Lint fixes were disabled [6]. Analyzed programs were never executed.

Pass	Source	Suppression comments
A	Original	Respected
B	Original	Ignored
C	Formatted	Respected
D	Formatted	Ignored

I ran all four checks because normal lint output cannot reveal warnings that a comment hides. Passes B and D expose those underlying warnings. Comparing A with B, and C with D, identifies which warnings are suppressed within each source snapshot. The visible warnings must be a multiset subset of the underlying warnings.

3.2 Syntax and warning correspondence

I compared abstract syntax trees, or ASTs, to check that formatting preserved the parsed program structure. Parsing included type comments, and the comparison omitted ordinary location attributes. This is a syntax check, not proof of identical runtime behavior. Some type-ignore line fields remain in the AST representation and can cause conservative exclusions.

The automatic matcher identifies warnings by rule, AST occurrence and token context. It does not match across formatting by line number or message alone. Correspondence must be unique, and the underlying warning identities must agree. A visible warning that becomes suppressed is Newly hidden; a suppressed warning that becomes visible is Newly exposed. Preserved means no visibility change for the selected warnings. Ambiguity, changed diagnostics, AST changes and tool failures have separate labels.

3.3 Inputs

I used 25 source templates, each with 8 directive modes, to make 200 controlled inputs. Modes include no directive, standalone comments, inline specific or blanket comments at different positions, and file-level suppression. Repeated executable structures across treatments are intentional. Input generation and validation are retained with the source manifest, and the design was fixed before the main run. These examples were designed to exercise formatting cases, rather than sampled from typical Python code.

The natural dataset contains 40 files from pinned commits of Requests, Flask and pytest. Eligible files were valid UTF-8 Python under the study grammar, contained a genuine `noqa` comment, met the size cap and passed the documented generated/vendored exclusions. Seeded hash ranking selected files within each repository without using their outcomes. The primary check used standardized settings and an artificial filename. A separate check used archived Ruff configuration and real relative paths where available. Neither recreated full project CI.

3.4 Analysis and supplementary review

Each input was tested with both formatters. Comparisons are paired by original source, and directive variants from one template are related. Detailed statistical choices were made after earlier counts were known, so the interval analyses are exploratory. Whole-cluster resampling keeps treatments and formatter pairing together [7]. Family-incidence Wilson calculations concern whether a family has any drift [8]. The model assumptions are not established by these authored inputs.

A later review examined every originally ambiguous comparison. Its separate procedure checked literal source occurrences, saved warnings and comment-only interventions. It proposed Preserved for all 32 comparisons. The reviewed calculations are conditional on accepting those proposals. The original labels and analysis remain available, and the matcher itself was not changed.

AI assistance was used for implementation, debugging, controlled input generation, analysis scripts and writing. Inputs were checked with deterministic tools, and measurements came from actual formatter and linter execution. The supplementary review was also AI-assisted; it was not independent human adjudication. Natural inputs were archived without study-generated edits, and upstream AI use is unknown.

4. Results

4.1 Controlled comparisons

Black produced 68 confirmed suppression changes and Ruff 35. Both were tested on the same 200 inputs, giving 400 controlled comparisons in total. Across all scheduled comparisons, the rates were 34.0% and 17.5%. No controlled comparison had an execution failure, AST change, underlying-diagnostic change or mixed drift.

Outcome	Original Black	Original Ruff	Reviewed Black	Reviewed Ruff
Preserved	116	149	132	165
Newly hidden	50	14	50	14
Newly exposed	18	21	18	21
Ambiguous	16	16	0	0

Each column has 200 comparisons. The reviewed columns use supplementary proposals, not new automatic classifications. Figure 1 shows the same distinction.

The original matcher resolved 184 input pairs. Among those inputs, the drift rates were 36.96% for Black and 19.02% for Ruff, a difference of 17.93 percentage points. The supplementary calculation includes 200 pairs and gives a difference of 16.50 points. The confirmed-change counts do not increase; the extra pairs are reviewed as no drift for both formatters.

Paired drift status	Ruff no drift	Ruff drift
Black no drift	132	0
Black drift	33	35

This is the supplementary 200-pair table. Binary drift overlap does not imply that both formatters changed visibility in the same direction.

4.2 Ambiguity and uncertainty

The ambiguity came from two template families. The 16 f-string comparisons had identical original and formatted bytes, but the embedded identifier did not align with a complete Python 3.11 token. The 16 exception-binding comparisons changed only assignment spacing. The matcher did not support the string-valued `ExceptHandler.name` field. The review found no visibility transition in either group. It checked all 128 saved lint replays and 64 comment interventions. Human review flags remain in the audit, particularly for the exception-binding interpretation.

Cluster model	Original difference interval (pp)	Reviewed difference interval (pp)
Template families	8.70 to 28.26	8.00 to 25.50
Syntax categories	5.92 to 32.14	4.89 to 30.29

These are exploratory 95% model intervals, not ecosystem confidence intervals. Each model used 20000 paired whole-cluster draws. Original calculations used 23 complete families and 12 categories; reviewed calculations use 25 and 13. The reviewed intervals were recalculated on their own denominator. Figure 2 shows the reviewed profiles and comparison. Original unresolved-outcome bounds and family-level Wilson results remain in their separate analysis files.

4.3 Source patterns

Directive placement and source layout were associated with different outcomes in the controlled suite. For example, Black collapsed a short list containing an inline specific `noqa` comment, placing both names under the comment at the end of the line. The warning for the second name became hidden. Ruff kept that example expanded and preserved visibility. Splitting semicolon-separated assignments exposed a previously suppressed warning with both formatters.

The pattern tables count the fixed treatments, not independent discoveries. Figure 3 shows drift counts by category and directive placement. Its boundary-category cells use the supplementary review; the original unresolved cells remain in the original figure.

Formatting changed text in 133 Black comparisons and 118 Ruff comparisons. Of those, the original analysis classified 57 and 75 as Preserved. With the review applied, those counts become 65 and 83. A text change alone therefore does not establish changed suppression.

4.4 Natural-source comparisons

No confirmed suppression changes were found in the 80 primary comparisons involving 40 original files. The natural-source results were not reclassified by the ambiguity review.

Repository	Files tested	Black Preserved / AST excluded	Ruff Preserved / AST excluded
Requests	7	7 / 0	7 / 0
Flask	13	12 / 1	13 / 0
pytest	20	18 / 2	19 / 1

There were 65 unchanged outputs and 15 changed outputs. Among the original files, 22 had no underlying warnings from the selected rules, corresponding to 44 comparisons. This left fewer opportunities to observe suppression drift than in the warning-focused controlled suite. The natural relevance figure is retained separately in the original Figure 4.

The 4 AST exclusions arose solely from shifted `TypeIgnore.lineno` fields. A secondary normalization made those ASTs equal, but the original labels remain excluded. Available native-configuration checks completed 66 comparisons and retained the same primary outcome classifications. Requests supplied 14 missing-configuration records, not completed native checks.

The early prospective natural repeat sample contained only unchanged outputs. The later full reproduction covered all 80 primary comparisons, including all 15 changed outputs. Those checks matched the originals. Repeated measurements confirm agreement on these inputs; they do not enlarge the independent sample.

5. Discussion

Black changed suppression more often than Ruff on the controlled cases. That difference describes this suite and these tool versions. It does not show that one formatter is generally better. Both hiding and exposure occurred, and fewer total changes did not mean fewer exposed warnings.

The list and semicolon examples show why checking underlying warnings mattered. In each case, the lint finding still referred to the same code occurrence, while its visible state changed. Whether that change violates an intended formatter policy is a separate question. The experiment measured the change, not the correctness of every warning or directive.

The repository sample produced a different result. Most outputs did not change under formatting, and many inputs had no warnings relevant to the selected rules. These observations may help explain the absence of confirmed drift, but they do not prove that prior formatting caused the result or that the formatters are safe on other files. If I repeated this part of the study, I would define a separate sample of files with selected warnings and changed formatter output. Its results would apply to that sample, not to all Python files.

The supplementary review identifies two specific gaps in the matcher. It does not establish a general solution for matching f-string or exception-binding warnings. If the matcher were extended, I would evaluate those changes on held-out examples with independently checked labels before claiming improved accuracy.

6. Limitations

The controlled examples were designed to stress selected layouts. Reusing a template across directive modes creates related observations. Whole-cluster resampling preserves some of that dependence, but the assumptions that clusters are independent and exchangeable remain unverified. Interval choices were made after earlier counts were known. The reviewed intervals also depend on the proposed labels and do not account for errors in the AI-assisted review.

The natural sample came from a few historical repositories chosen for feasibility. Selection required a natural `noqa` comment, valid UTF-8 source and the study grammar, and excluded files using documented size and generated/vendored rules. The sample cannot establish a Python-wide rate. Empty selected-warning sets and unchanged outputs further limit the negative finding. Counts and bounds for each eligible historical archive remain separate in the original analysis.

The AST gate is conservative and is not a proof of runtime equivalence. Warning matching covers only the selected Ruff rules and supported anchors. The same AI-assisted process developed code, examples, tests and interpretation. Development controls and repeated executions reduce some implementation risks but do not supply independent held-out ground truth. I therefore report no precision, recall or F1.

Only the saved Windows/Python environment and pinned tool versions were tested. The standardized filename differs from real project context, and native configuration checks are partial. A fresh-machine installation, other platforms, later releases, other diagnostic families and complete project CI were not evaluated.

7. Conclusion

Formatting changed which Ruff warnings were suppressed in the controlled tests, even when the strict AST check passed. No confirmed changes appeared in the natural-source sample. The separate review supports Preserved proposals for the original matcher ambiguities while leaving the automatic record intact.

The next useful work is independent review of a held-out set, a larger natural sample with relevant warnings and changed source, and comparisons across additional rules and tool versions. A small developer check could then be evaluated against that evidence. These are proposed extensions, not completed features.

8. References

[1] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. 2025. An Empirical Study of Suppressed Static Analysis Warnings. Proceedings of the ACM on Software Engineering 2(FSE), Article FSE014. DOI. Published author text.

[2] Junjie Li and Jinqiu Yang. 2024. Tracking the Evolution of Static Code Warnings: The State-of-the-Art and a Better Approach. IEEE Transactions on Software Engineering 50(3), 534-550. DOI. Author text.

[3] William M. McKeeman. 1998. Differential Testing for Software. Digital Technical Journal 10(1), 100-107. Original journal.

[4] Black contributors. Frequently Asked Questions, version 25.1.0. Versioned source.

[5] Astral contributors. Known Deviations from Black, Ruff 0.11.13. Versioned source.

[6] Astral contributors. Ruff linter and formatter documentation. Current pages provide background; retained pinned help and actual commands govern the experiment.

[7] StataCorp. Bootstrap resampling reference manual, cluster resampling. Manual. The calculation here used NumPy, not Stata.

[8] NIST/SEMATECH. e-Handbook of Statistical Methods, confidence limits for a proportion. Wilson method.

Data and reproduction

The reproduction instructions identify the pinned environment and commands. Original data and metrics are under `datasets` and `reports/phase5/analysis`. The supplementary audit retains each source, raw diagnostic output and review reason. Reviewed metrics and draws are under `reports/publication-phase-a/analysis`. A separate claim ledger for this public draft is under `reports/publication-phase-b`.

This draft does not replace the original report or its PDF. Scientific findings and decoded source bytes are preserved in this privacy-derived snapshot. Execution metadata is anonymized; full upstream archives and unrelated assets are excluded. See the snapshot notes and third-party notices. The MIT license covers author-owned code and documentation; upstream terms remain separate. No publication has been performed.